

Nix

29 marzo 2026

Indice

1	Intro	5
1.1	Strumenti utili	6
1.2	Tipi di dati principali	6
1.2.1	Roba classica	6
1.2.2	Paths	7
1.2.3	Dati composti	8
1.3	Cose inerenti variabili	8
1.3.1	Definizione di variabili	8
1.3.2	Accesso a variabili di un set	9
1.3.3	Aggiunta di variabili a set o lista	9
1.4	Funzioni	9
1.5	Controllo del flusso	10
1.6	Builtins	10

Capitolo 1

Intro

Nix e' un linguaggio di programmazione funzionale e un package manager che crea pacchetti locati

```
/nix/store/q06x3j1l2yfczckz2bzqak089p43ixkkq-librewolf-33.1/
.
|-- bin
|   '-- librewolf
|-- lib
|   |-- librewolf
|   '-- mozilla
'-- share
    |-- applications
    '-- icons
```

dove `b6gvzjyb2pg0` e' un identificatore unico dipacchetto/dipendente (hash crittografico del grafico di dipendenze del pacchetto).

È possibile avere

- piu versioni dello stesso pacchetto allo stesso momento, ciascuna avra' la propria cartella con dipendenze
- aggiornamenti atomici: nuove versioni non sovrascrivono le vecchie e non vi sono momenti dell'aggiornamento in cui il sistema e' inutilizzabile

Il linguaggio serve per creare derivations, ossia ricette di come file esistenti (eg nel web) siano utilizzati per creare altri files.

È un linguaggio

- domain specifico per la creazione di derivazioni
- dichiarativo: non vi e' una sequenza di step, le dipendenze tra operazioni sono stabilite solamente attraverso i dati
- puro: a parita di input vi e' parita di output
- funzionale: funzioni first class citizen
- coi tipi di dato dinamici (a-la python), errori di tipo vengono identificati solo in sede di esecuzione

- lo spazio e l'indentazione non hanno significato
- si commenta con cancelletto

1.1 Strumenti utili

Per

- utilizzare la repl:

```
nix repl
```

dalla quale

```
:?          # help
:doc <expr>  # documentazione su builtin: es  :doc builtins.add
builtins     # stampa il set di builtins
:p espressione # stampa l'espressione per esteso
:q          # esce
```

- eseguire un file.nix

```
nix-instantiate --eval file.nix
```

In ogni file .nix vi deve esser una espressione nix.

1.2 Tipi di dati principali

1.2.1 Roba classica

Sono riportati in seguito

```
null          #valore null

# -----
# Stringhe
# -----
"hello wolrd"  # stringa

'',           # stringa multilinea
  stringa multi
  linea
'',

"foo" + "bar"  # concatenazione

"${variabile}" # intepolazione di stringa con variabile

# -----
# numeri
```

```
# -----

123                # intero

12.34              # float

# -----
# bool
# -----

true, false,      # booleani

!true             # not
bool1 && bool2     # and
bool1 || bool2    # or

bool1 -> bool2     # logical implication
                  # Equivalent to:
                  # - !bool1 || bool2
                  # - if bool1 then bool2 else true
```

1.2.2 Paths

```
# -----
# paths
# -----

/etc                # path assoluto

~/config            # path assoluto con espansione home dell'utente

./foo.png           # path relativo: path riconoscibili, foo.png e basta
                  # non basta

<nixpkgs>           # path di lookup. Valore determinato da $NIX_PATH
                  # eg
                  # nixpkgs=/nix/var/nix/profiles/per-user/root/channels/nixos
```

A file is not required to exist at a given path in order for that path value to be valid, but a path that is converted to a string with string interpolation or string-and-path concatenation must resolve to a readable file or directory.

I path possono essere interpolati/sostituiti con variabili

```
let foo="foo"; bar="qwe"; in ./a.${foo}/b.${bar}
/nix/store/x3g5fh0i9rlg7z97pgs0g8crxmihkk8k-nixos-25.11/nixos/pkgs/a.foo/b.qwe
```

1.2.3 Dati composti

Per attributes set si pongono tra graffe e l'assegnazione ad ogni elemento appartenente termina con `;`. Agli attributi si accede con `.`

```
{ x = 1; y = 2; } # attribute set (flat)

{ foo.bar = 1; } # nested set, equivalente a { foo = { bar = 1; }; }

rec {
    x = "foo";      # set ricorsivo equiv a { x = "foo"; y = "foobar"; }
    y = x + "bar";  # senza rec x alla seconda riga si riferisce
                    # ad un x eventualmente nell'environment padre
}                  # inoltre non vi è ordine nell'inserimento e le istruzioni
                    # entro graffe potrebbero essere invertite

{ x = 1; y = 2; }.x      # selezione di elemento da set (valuta 1)
{ x = 1; y = 2; }.z or 3 # selezione con default (valuta 3)
{ x = 1; y = 2; } // { z = 3; } # merging di due set

let bar = "foo"; in      # supporto per interpolazione nell'assegnazione
{ ${bar} = 123; }.foo    # degli attributi (valuta 123)

let bar = "foo"; in      # supporto interpolazione nell'accesso ad
{ foo = 123; }.${bar}    # attributi (valuta 123)
```

Liste Liste

```
[ "foo" "bar" "baz" ] # liste di tre elementi
[ 1 2 3 ]

[ 123 ./foo.nix "abc" (f { x = y; }) ] # lista con 4 elementi,
                                         # ultimo risultato di chiamata

[ 123 ./foo.nix "abc" f { x = y; } ]    # lista con 5 elementi: quarto e'
                                         # variabile f (probabilmente funzione)
                                         # il quinto un set
```

1.3 Cose inerenti variabili

1.3.1 Definizione di variabili

Si definiscono in un `let-in` e si utilizzano nella prima espressione dopo `in`

```
let
    x = "foo";
    y = "bar";
in x + y
```

1.3.2 Accesso a variabili di un set

`builtins` sotto e' un set (di funzioni), con `with` aggiungiamo tutti gli attributi di un set al presente scope (ossia possiamo chiamare `builtins.head` senza preporre `builtins`). `head` poi e' una funzione che prende in input il vettore e restituisce 1

```
# with e1; e2
with builtins; head [ 1 2 3 ]
```

L'utilizzo comune di `with` è con `import`

```
with (import ./definitions.nix); ..
```

1.3.3 Aggiunta di variabili a set o lista

all'interno di un set o di un `let`, `inherit` aggiunge variabili allo scope prendendole da qualche altra parte

```
inherit pkgs src;      # equivalente di pkgs = pkgs; src = src;
                       # prende dall'enclosing environment

inherit (pkgs) lib stdenv; # tra tonde un set equivale a
                           # lib=pkgs.lib;
                           # stdenv=pkgs.stdenv;
```

1.4 Funzioni

Le funzioni prendono un solo argomento: per funzioni che ne ammettano di piu usare un set come argomento o definire la funzione ricorsivamente

```
x: x + 1                # definizione al volo di lambda (funzioni)

x: y: x + y             # definizione ricorsiva di funzione con due parametri
                       # equivale a x: (y: x + y)

{ x, y }: x + y         # definizione con set (sia x che y necessari in)
                       # chiamata

{ x, y ? "bar" }: x + y # uso di valore default "bar" y (opzionale),
                       # mentre x e' necessario

{ x, y, ... }: x + y    # eventuali altri parametri oltre a x e y (richiesti)
                       # sono ignorati

{ x, y } @ args: x + y  # in entrambi i casi funzione che attende un set
args @ { x, y }: x + y  # con x e y e binda l'intero set ad args?
```

Per chiamate si pone il corpo della funzione seguito da uno spazio e il parametro

```

(x: x + 1) 2      # chiamata (implicita), valuta 3

let              # definizione e assegnazione ad una variabile
  f = x: x + 1;   # e chiamata per nome
in f (f (f 1))

Programmazione funzionale

map (x: x + x) [ 1 2 3 ]    # applica una funzione a tutti gli elementi
                             # di una lista (restituendo una lista) [2 4 6]

mapAttrs (nam: val: val*10) { a = 1; b = 2; } # applica una funzione a tutti gli ele
                                                # di un set (restituendo una set) [2 4]

let
  concat = x: y: x + y;
in
  map (concat "foo") [ "bar" "bla" ] # ["foobar" "foobla"]

# pipe (sperimentale)
a |> b    # equivalente ad b a. es 1 |> builtins.add 2 |> builtins.mul 3
          # valuta 9

\end{verbatim*}

\section{Loading di codice esterno}
Mediante import: carica/valuta/restituisce l'espressione nix nel file dato
\begin{verbatim*}
import ./foo.nix

```

1.5 Controllo del flusso

ifelse classico

```

# if e1 then e2 else e3
if 1 + 1 == 2 then "yes!" else "no!"    # valuta

```

Assertzioni usate per controllare requisiti: se `e1` valuta vero si ritorna `e2`, altrimenti errore

```

# assert e1; e2
assert 1 + 1 == 2; "yes!"

```

1.6 Builtins

Le builtins sono contenute nel set `builtin` e disponibili come `builtins.nome`,

```

builtins      # nella REPL stampa tutte le builtins

## check di tipo dei dati
isAttrs {x=1; y=2;}

```

```
isBool true
isFloat 3.14
isFunction (x: x+1)
isInt 3
isList [1 2 3]
isNull
isPath /etc/nixos
isString "asd"
```

```
# set
attrNames set      # restituisce lista con nomi assegnati in un set
attrValues set     # restituisce lista con valori
hasAttr attributo set # verifica se ha attributo
getAttr attributo set # estrae il valoreun dato attributo
                    # builtins.getAttr "a" {a=1; b=2;}
                    # valuta 1
intersectAttrs s1 s2 # intersezione di due set (solo sulla base del nome non del contenuto deg
                    # attributi)
removeAttrs set list # restituisce un set dove i nomi di list sono tolti dagli elementi di set
```

```
# list
length l          # lunghezza lista
concatLists lists # Concatenate a list of lists into a single list.
elem x lista      # true se x è in lista
```

```
# path
baseNameOf path   # basename di un path (o stringa)
pathExists path
```

```
# funzioni e programmazione funzionale
functionArgs f    # ritorna set con i nomi degli argomenti attesi
partition f list  # applica f (predicato) alla lista e ritorna un set con "right"
                  # (elementi con predicato vero) e "wrong" (falso)
groupBy f list    # ritorna un set che raggruppa i valori di list sulla base dell'applicazione d
                  # builtins.groupBy (builtins.substring 0 1) ["foo" "bar" "baz"]
                  # { b = [ "bar" "baz" ]; f = [ "foo" ]; }
map f list        # applica la funzione a tutti gli elementi della lista (restituisce lista)
mapAttrs f set    # applica f ad ogni elemento del set (restituisce set)
sort f list       # sorta gli elementi di una lista basati sulla funzione f (comparatore)
all function list  # ritorna true se function applicata alla lista da tutti trye
any function list  # any ..
```

```
# generazione di dati
genlist f lunghezza # genera lista lunga "lunghezza" applicando f a [0 1 .. (lunghezza -1)
```

```
# elaborazione stringhe/testo
toString x          # converte a stringa
stringLength e      # lunghezza stringa
```

```
replaceStrings from to s # ricerca e rimpiazzo stringhe
split regex str          # splitting
splitVersion s           # splitting di versione software
substring start len s    #

# io
fetchGit args            # builtins.fetchGit https://github.com/lbraglia/lbmisc.git
fetchTarball args        # scarica un targz
fetchTree arg            # fetcha qualunque cosa ma è da abilitare
import path              # valuta e ritorna l'espressione contenuta nel file/path
                        # se il path è una directory, viene valutata default.nix
toFile s testo           # scrive testo su s nel lath

# errori e simili
throw msg                # utilizzato se qualcosa va male per dare errore
```

TODOHERE sono arrivato a <https://nix.dev/manual/nix/2.34/language/derivations>